

งานปฏิบัติการกลุ่ม: Queue Algorithm Design & Analysis with Java

หัวข้อหลัก: Queue, Circular Queue, Priority Queue, Deque และการประยุกต์ใช้

กลุ่มที่ 7: Ride-Hailing Driver Assignment — ระบบจับคู่ผู้โดยสารกับคนขับ

Case Story

ระบบเรียกรถมีผู้โดยสารส่งคำขอเข้ามาเป็น Queue และมี Driver ที่พร้อมรับงาน โดยระบบต้องจับคู่ผู้โดยสาร (Passenger Request) กับคนขับ (Driver) ให้เหมาะสมที่สุดตามเงื่อนไขของแต่ละ Algorithm

Scenario ที่ใช้ในรายงานนี้

Passengers: R1, R2, R3, R4 Drivers: D1, D2, D3

	D1	D2	D3
R1	5	2	8
R2	1	6	4
R3	7	3	2
R4	3	5	6

สมมติฐานเพิ่มเติม: ลำดับการ request คือ $R1 \rightarrow R2 \rightarrow R3 \rightarrow R4$ ตามลำดับที่แสดง (R1 มาก่อนสุด) และคนขับทั้ง 3 คนพร้อมรับงานตั้งแต่ต้น (Available ตั้งแต่เริ่มต้น) คนขับ 1 คนรับได้ 1 งานต่อรอบการจับคู่

ส่วนที่ 1: การวิเคราะห์ปัญหา (Problem Analysis)

Input

- รายการคำขอผู้โดยสาร (Passenger Request): Request ID, Location, Request Time, Distance ไปยังคนขับแต่ละคน
- รายการคนขับ (Driver): Driver ID, Location, Available Time
- Distance Matrix ระหว่างผู้โดยสารแต่ละคนกับคนขับแต่ละคน

Output

- คู่ที่จับคู่สำเร็จ (Passenger-Driver) พร้อมระยะทางของการจับคู่
- รายชื่อผู้โดยสารที่ยังไม่ได้รับคนขับ (ต้องรอคิวต่อ)
- ระยะทางรวมของคนขับทั้งหมด (Total Driver Distance) สำหรับใช้เปรียบเทียบ Algorithm

Constraints

- จำนวนคนขับอาจน้อยกว่าจำนวนคำขอ ทำให้มีผู้โดยสารบางคนต้องรอ (ไม่ได้รับคนขับในรอบนั้น)
- คนขับ 1 คน รับผิดชอบได้ 1 งานเท่านั้นในรอบการจับคู่ปัจจุบัน (ไม่มีการจองซ้อน)
- ต้องประมวลผลคำขอโดยเคารพลำดับก่อน-หลังของการเข้ามา (First-Come ต้องมีสิทธิ์ก่อนเสมอในการ 'เข้าคิวเลือก')
- ระยะทางระหว่างผู้โดยสารและคนขับเป็นค่าคงที่ที่ทราบล่วงหน้า (Distance Matrix)

Queue เก็บข้อมูลอะไร

Queue หลักของระบบเก็บ "คำขอผู้โดยสาร (Passenger Request)" เรียงตามลำดับเวลาที่เข้ามา (arrival order) เพื่อรับประกันว่าใครขอก่อนจะได้รับสิทธิ์ประมวลผลก่อน ส่วนอีก Queue หนึ่งเก็บ "คนขับที่พร้อมรับงาน (Available Driver)"

Queue ชนิดใดเหมาะสม

- Algorithm A (FIFO Matching): ใช้ Queue ธรรมดาแบบ FIFO (Queue<Passenger> และ Queue<Driver>) เพราะเกตติกาคือจับคู่ตามลำดับก่อน-หลังล้วนๆ ไม่ต้องจัดลำดับความสำคัญใดๆ เพิ่มเติม
- Algorithm B (Nearest Driver Matching): ใช้ Priority Queue (PriorityQueue<Driver>) เพราะสำหรับผู้โดยสารแต่ละคน ต้องเลือกคนขับที่ "ระยะทางน้อยที่สุด" ก่อนเสมอ — priority คือระยะทาง ยิ่งน้อยยิ่งถูกดึงออกก่อน

เหตุผลที่ต้องใช้ Queue

คำขอผู้โดยสารในระบบเรียกรถเข้ามาแบบต่อเนื่องทีละคน (stream) โดยไม่ทราบจำนวนล่วงหน้า และระบบต้องประมวลผลแบบอัตโนมัติตามกติกาที่กำหนดไว้ตายตัว (FIFO หรือเรียงตามระยะทาง) จึงจำเป็นต้องใช้โครงสร้างข้อมูลที่รองรับการเพิ่มข้อมูลเข้า (enqueue/offer) และดึงข้อมูลออกมาประมวลผลตามกติกา (dequeue/poll) ได้อย่างมีประสิทธิภาพ ซึ่ง Queue และ Priority Queue ตอบโจทย์นี้โดยตรง

ส่วนที่ 2: Queue Design และ Algorithm Design (Pseudocode)

Queue Design

- requestQueue : Queue<Passenger> — เก็บผู้โดยสารเรียงตามเวลาที่ request เข้ามา (ใช้ร่วมกันทั้งสอง Algorithm)
- driverQueue (Algorithm A) : Queue<Driver> — เก็บคนขับเรียงตามลำดับที่พร้อมรับงาน
- nearestPQ (Algorithm B) : PriorityQueue<Driver> — สร้างขึ้นใหม่ทุกครั้งที่มีผู้โดยสาร 1 คนถูก dequeue ออกมา โดยจัดลำดับความสำคัญ (priority) จากระยะทางของคนขับที่ยังว่างอยู่ไปยังผู้โดยสารคนนั้น จากนั้นน้อยไปมาก

Pseudocode Algorithm A: FIFO_MATCHING

```
Algorithm FIFO_MATCHING
requestQueue <- enqueue ผู้โดยสารทุกคนตามลำดับเวลาที่ request
driverQueue  <- enqueue คนขับทุกคนตามลำดับที่พร้อมรับงาน

while requestQueue is not empty AND driverQueue is not empty
  p <- dequeue(requestQueue)
  d <- dequeue(driverQueue)
  distance <- distanceMatrix[p][d]
  บันทึกผล match(p, d, distance)
end while

while requestQueue is not empty          // ไม่มีคนขับเหลือ
  p <- dequeue(requestQueue)
  บันทึกผล unmatched(p)
end while
end Algorithm
```

Pseudocode Algorithm B: NEAREST_DRIVER_MATCHING

```
Algorithm NEAREST_DRIVER_MATCHING
requestQueue <- enqueue ผู้โดยสารทุกคนตามลำดับเวลาที่ request
availableDrivers <- เซตของคนขับทั้งหมดที่พร้อมรับงาน

while requestQueue is not empty
  p <- dequeue(requestQueue)
  pq <- PriorityQueue วาง เรียงตามระยะทางไปยัง p (น้อย -> มาก)

  for each d in availableDrivers
    offer(pq, d, distanceMatrix[p][d])
  end for

  if pq is not empty
    d <- poll(pq)          // คนขับที่ใกล้ p ที่สุด
    ลบ d ออกจาก availableDrivers
    บันทึกผล match(p, d, distanceMatrix[p][d])
  else
    บันทึกผล unmatched(p)
  end if
end while
end Algorithm
```

ส่วนที่ 3: Queue Trace (อย่างน้อย 10 Operations)

แสดงการ trace การทำงานของ Algorithm A (FIFO Matching) กับ Scenario R1-R4 / D1-D3 เนื่องจากเห็นการ enqueue/dequeue ของทั้งสอง Queue ได้ชัดเจนที่สุด

Step	Operation	requestQueue	driverQueue	Output
1	ENQUEUE R1 -> requestQueue	[]	[]	-
2	ENQUEUE R2 -> requestQueue	[R1]	[]	-
3	ENQUEUE R3 -> requestQueue	[R1,R2]	[]	-
4	ENQUEUE R4 -> requestQueue	[R1,R2,R3]	[]	-
5	ENQUEUE D1 -> driverQueue	[R1,R2,R3,R4]	[]	-
6	ENQUEUE D2 -> driverQueue	[R1,R2,R3,R4]	[D1]	-
7	ENQUEUE D3 -> driverQueue	[R1,R2,R3,R4]	[D1,D2]	-
8	DEQUEUE ทั้งสอง Queue	[R2,R3,R4]	[D2,D3]	match(R1,D1) dist=5
9	DEQUEUE ทั้งสอง Queue	[R3,R4]	[D3]	match(R2,D2) dist=6
10	DEQUEUE ทั้งสอง Queue	[R4]	[]	match(R3,D3) dist=2
11	driverQueue ว่าง -> DEQUEUE requestQueue	[]	[]	unmatched(R4)

เหตุใด Queue จึงเปลี่ยนสถานะดังกล่าว: ทุก ENQUEUE จะเพิ่มสมาชิกใหม่ต่อท้าย Queue (rear) โดยไม่กระทบสมาชิกเดิม ส่วนทุก DEQUEUE จะดึงสมาชิกที่อยู่หน้าสุด (front) ออกเสมอ ซึ่งคือสมาชิกที่เข้ามาก่อนสมาชิกอื่นทั้งหมดที่ยังอยู่ใน Queue — คุณสมบัตินี้ทำให้มั่นใจได้ว่า R1 (มาก่อนสุด) จะถูกจับคู่กับ D1 (พร้อมก่อนสุด) เสมอ ไม่ว่าระยะทางจะเป็นเท่าใด และเมื่อ driverQueue ว่างลง (step 11) ผู้โดยสารที่เหลือ (R4) จะยังคงค้างอยู่ใน requestQueue รอรอบถัดไป

ส่วนที่ 4: Correctness / Invariant

Algorithm A: FIFO Matching

Loop Invariant: ก่อนเริ่มแต่ละรอบของการวนซ้ำ สมาชิกที่อยู่บริเวณ front ของ requestQueue คือผู้โดยสารที่ request เข้ามาก่อนผู้โดยสารอื่นทั้งหมดที่ยังไม่ได้รับการจับคู่ และสมาชิกที่อยู่บริเวณ front ของ driverQueue คือคนขับที่พร้อมรับงานก่อนคนขับอื่นที่ยังว่างอยู่ทั้งหมด ดังนั้นทุกคู่ที่ถูกจับคู่ในแต่ละรอบจึงรักษาคุณสมบัติ First-In-First-Out ไว้เสมอ นั่นคือลำดับการได้รับการบริการจะตรงกับลำดับการ request 100%

Algorithm B: Nearest Driver Matching

Loop Invariant: ก่อนเริ่มแต่ละรอบ PriorityQueue ที่สร้างขึ้นสำหรับผู้โดยสารคนปัจจุบันจะบรรจุคนขับที่ยังว่างอยู่ทั้งหมด ณ ขณะนั้น เรียงลำดับความสำคัญตามระยะทางจากน้อยไปมาก ดังนั้นสมาชิกที่ถูก poll() ออกมาคือคนขับที่มีระยะทางน้อยที่สุดในบรรดาคนขับที่เหลืออยู่เสมอ (Greedy ที่ optimal เฉพาะในระดับผู้โดยสารแต่ละคน แต่ไม่รับประกันผลรวมที่ optimal ที่สุดของทั้งระบบ)

การยุติของ Algorithm (Termination)

ทั้งสอง Algorithm รับประกันการยุติการทำงานเสมอ เพราะขนาดของ requestQueue ลดลง 1 หน่วยทุกครั้งที่มีการ dequeue (ไม่ว่าจะจับคู่สำเร็จหรือไม่) และ requestQueue เริ่มต้นมีขนาดจำกัด (n) ดังนั้นการวนซ้ำจะสิ้นสุดภายในจำนวนรอบจำกัดเสมอ

ส่วนที่ 5: Time Complexity

Complexity ของ Operation พื้นฐาน

Operation	โครงสร้างข้อมูล	Time Complexity
enqueue / offer	Queue (ArrayDeque)	$O(1)$
dequeue / poll	Queue (ArrayDeque)	$O(1)$
peek	Queue (ArrayDeque)	$O(1)$
offer	PriorityQueue	$O(\log n)$
poll	PriorityQueue	$O(\log n)$
peek	PriorityQueue	$O(1)$

Algorithm A: FIFO Matching

- enqueue ผู้โดยสารทั้งหมด n คน: $O(n)$
- enqueue คนขับทั้งหมด m คน: $O(m)$
- รอบจับคู่: ทำ $\min(n, m)$ ครั้ง แต่ละครั้ง dequeue สองครั้ง $O(1)$: รวม $O(\min(n, m))$
- รวมทั้งหมด: $O(n + m)$ — เป็นเชิงเส้นตามจำนวนผู้โดยสารและคนขับ

Algorithm B: Nearest Driver Matching

- enqueue ผู้โดยสารทั้งหมด n คน: $O(n)$
- สำหรับผู้โดยสารแต่ละคน (n รอบ) ต้องสร้าง PriorityQueue ใหม่จากคนขับที่ยังว่าง (สูงสุด m คน) โดย offer ทีละตัว $O(\log m)$ ต่อครั้ง: $O(m \log m)$ ต่อผู้โดยสาร 1 คน
- รวมทั้งหมด (worst case): $O(n \cdot m \log m)$ — แย่กว่า FIFO ชัดเจน
เพราะต้องเปรียบเทียบระยะทางใหม่ทุกครั้งที่มีผู้โดยสารใหม่เข้ามา

สรุป: Algorithm A เร็วกว่าและ scale ได้ดีกว่ามากเมื่อ n และ m มีขนาดใหญ่ ในขณะที่ Algorithm B แลกความเร็วกับคุณภาพของผลลัพธ์ (ระยะทางรวมต่ำกว่า)

ส่วนที่ 6: Space Complexity

- Distance Matrix ขนาด $n \times m$ ต้องใช้พื้นที่ $O(n \cdot m)$ เป็น input ที่ให้มา (ไม่นับเป็น auxiliary space ของ Algorithm)
- requestQueue เก็บสูงสุด n รายการ: $O(n)$
- driverQueue หรือ availableDrivers เก็บสูงสุด m รายการ: $O(m)$

- Algorithm B ต้องสร้าง PriorityQueue ใหม่ทุกครั้ง แต่ใช้พื้นที่ไม่เกิน $O(m)$ ต่อรอบ (ไม่สะสม เพราะสร้างใหม่และปล่อยทิ้งทุกครั้ง)

สรุป Auxiliary Space Complexity ของทั้งสอง Algorithm: $O(n + m)$

ส่วนที่ 7: Java Implementation

ใช้ Queue<T> (ArrayDeque) สำหรับ Algorithm A และ PriorityQueue<T> สำหรับ Algorithm B ตามที่ออกแบบไว้ในส่วนที่ 2 ได้ดัดแปลงเพิ่มเติมแสดงด้านล่าง (ไฟล์ RideHailingQueueAlgorithm.java แนบมาพร้อมรายงานนี้)

```
import java.util.*;

/**
 * กลุ่มที่ 7: Ride-Hailing Driver Assignment
 * ออกแบบด้วย Queue<T> และ PriorityQueue<T> ตามหลักการ Queue Algorithm Design
 *
 * Algorithm A: FIFO Matching      -> Queue<Passenger> + Queue<Driver>
 * Algorithm B: Nearest Driver Match -> Queue<Passenger> + PriorityQueue<Driver> (ต่อผู้โดยสาร 1 คน)
 */
public class RideHailingQueueAlgorithm {

    static class Passenger {
        String id;
        int index; // แถวใน distanceMatrix
        Passenger(String id, int index) { this.id = id; this.index = index; }
        public String toString() { return id; }
    }

    static class Driver {
        String id;
        int index; // คอลัมน์ใน distanceMatrix
        Driver(String id, int index) { this.id = id; this.index = index; }
        public String toString() { return id; }
    }

    static class MatchResult {
        String passengerId;
        String driverId; // null = ยังไม่ได้จับคู่ (รอคิว)
        int distance; // -1 = ไม่มีค่า
        MatchResult(String p, String d, int dist) {
            passengerId = p; driverId = d; distance = dist;
        }
    }

    public static void main(String[] args) {
        String[] passengerIds = {"R1", "R2", "R3", "R4"};
        String[] driverIds    = {"D1", "D2", "D3"};

        int[][] distanceMatrix = {
            {5, 2, 8}, // R1
            {1, 6, 4}, // R2
            {7, 3, 2}, // R3
        }
    }
}
```

```

        {3, 5, 6}, // R4
    };

    List<Passenger> passengers = new ArrayList<>();
    for (int i = 0; i < passengerIds.length; i++) passengers.add(new
Passenger(passengerIds[i], i));

    List<Driver> drivers = new ArrayList<>();
    for (int i = 0; i < driverIds.length; i++) drivers.add(new Driver(driverIds[i],
i));

    System.out.println("=== Algorithm A: FIFO Matching ===");
    List<MatchResult> fifo = fifoMatching(passengers, drivers, distanceMatrix);
    printResults(fifo);

    System.out.println("\n=== Algorithm B: Nearest Driver Matching ===");
    List<MatchResult> nearest = nearestMatching(passengers, drivers, distanceMatrix);
    printResults(nearest);
}

/**
 * Algorithm A: FIFO Matching
 * ใช้ Queue<Passenger> และ Queue<Driver> (ArrayDeque) ล้วน ๆ
 * dequeue คู่กันไปเรื่อย ๆ จนกว่า Queue ใด Queue หนึ่งจะว่าง
 */
static List<MatchResult> fifoMatching(List<Passenger> passengers, List<Driver> drivers,
int[][] dist) {
    Queue<Passenger> requestQueue = new ArrayDeque<>(passengers);
    Queue<Driver> driverQueue = new ArrayDeque<>(drivers);

    List<MatchResult> results = new ArrayList<>();

    while (!requestQueue.isEmpty() && !driverQueue.isEmpty()) {
        Passenger p = requestQueue.poll(); // dequeue
        Driver d = driverQueue.poll(); // dequeue
        int distance = dist[p.index][d.index];
        results.add(new MatchResult(p.id, d.id, distance));
    }

    // ผู้โดยสารที่เหลือ (ไม่มีคนขับพอ) ถือว่ารอคิว
    while (!requestQueue.isEmpty()) {
        Passenger p = requestQueue.poll();
        results.add(new MatchResult(p.id, null, -1));
    }

    return results;
}

/**
 * Algorithm B: Nearest Driver Matching
 * requestQueue เป็น Queue ปกติ (รักษาลำดับ First-Come)
 * แต่ต่อผู้โดยสาร 1 คน จะสร้าง PriorityQueue<Driver> ใหม่จากคนขับที่ยังว่าง
 * โดยจัดลำดับความสำคัญ (priority) ตามระยะทางไปยังผู้โดยสารคนนั้น (น้อย -> มาก)

```

```

*/
static List<MatchResult> nearestMatching(List<Passenger> passengers, List<Driver>
drivers, int[][] dist) {
    Queue<Passenger> requestQueue = new ArrayDeque<>(passengers);
    List<Driver> availableDrivers = new ArrayList<>(drivers); // pool คนขับที่ยังว่าง

    List<MatchResult> results = new ArrayList<>();

    while (!requestQueue.isEmpty()) {
        Passenger p = requestQueue.poll(); // dequeue ตามลำดับ First-Come

        final Passenger current = p;
        PriorityQueue<Driver> pq = new PriorityQueue<>(
            Comparator.comparingInt(d -> dist[current.index][d.index])
        );
        pq.addAll(availableDrivers); // offer คนขับที่ว่างทั้งหมดเข้า PriorityQueue

        if (!pq.isEmpty()) {
            Driver nearest = pq.poll(); // ดึงคนขับที่ระยะทางน้อยที่สุด
            availableDrivers.remove(nearest);
            int distance = dist[p.index][nearest.index];
            results.add(new MatchResult(p.id, nearest.id, distance));
        } else {
            results.add(new MatchResult(p.id, null, -1));
        }
    }

    return results;
}

/**
 * Cancel Case: ยกเลิกคำขอที่อยู่กลาง requestQueue (Test Case #6)
 * ArrayDeque.remove(Object) จะค้นหาและลบสมาชิกตัวแรกที่ตรงกัน O(n)
 * คืนค่า false ถ้าไม่พบ (เช่น ยกเลิกคำขอที่ไม่มีอยู่จริง) โดยไม่เกิด Exception
 */
static boolean cancelRequest(Queue<Passenger> requestQueue, String passengerId) {
    Passenger target = null;
    for (Passenger p : requestQueue) {
        if (p.id.equals(passengerId)) { target = p; break; }
    }
    if (target == null) return false; // ไม่พบ -> ไม่มีผลใด ๆ
    return requestQueue.remove(target);
}

static void printResults(List<MatchResult> results) {
    System.out.printf("%-10s%-10s%-10s\n", "Passenger", "Driver", "Distance");
    for (MatchResult r : results) {
        System.out.printf("%-10s%-10s%-10s\n",
            r.passengerId,
            r.driverId == null ? "-" : r.driverId,
            r.distance < 0 ? "รอคิว" : String.valueOf(r.distance));
    }
}
}

```


ส่วนที่ 8: Test Cases (อย่างน้อย 6 กรณี)

#	Test Case	รายละเอียด / Input	ผลลัพธ์ที่คาดหวัง
1	Normal Case	R1-R4, D1-D3 ตาม Distance Matrix ของ Scenario หลัก	FIFO: จับคู่ 3 คู่ (R1-D1,R2-D2,R3-D3) เหลือ R4 รอ / Nearest: จับคู่ 3 คู่ (R1-D2,R2-D1,R3-D3) เหลือ R4 รอ, ระยะทางรวมน้อยกว่า FIFO
2	Empty Queue	ไม่มีผู้โดยสารเลย (n = 0) แต่มีคนขับ 3 คน	requestQueue วางตั้งแต่ต้น ไม่มีการจับคู่เกิดขึ้น ไม่เกิด Exception, ผลลัพธ์เป็นลิสต์ว่าง
3	Single Item	ผู้โดยสาร 1 คน (R1), คนขับ 1 คน (D1)	จับคู่ทันที R1-D1 ทั้งสอง Algorithm ให้ผลเหมือนกัน (มีตัวเลือกเดียว)
4	Large Queue	จำลองผู้โดยสาร n = 10,000 คน, คนขับ m = 5,000 คน ด้วยระยะทางสุ่ม (fixed seed)	ทุกคู่ที่จับคู่ต้องมี distance ที่ถูกต้องตรงตาม matrix, จำนวนคู่ที่จับคู่ได้ = min(n,m), ไม่มี driver ถูกใช้ซ้ำ
5	Special/Edge Case	ทุกคู่ผู้โดยสาร-คนขับมีระยะทางเท่ากันหมด (Priority เท่ากันหมด)	Nearest Matching ต้องเลือกได้อย่างสม่ำเสมอ (deterministic ตามลำดับที่ offer เข้า PriorityQueue) ไม่เกิด error หรือวนลูปไม่รู้จบ
6	Cancel Case	ยกเลิกคำขอ R2 ขณะที่ยังอยู่กลาง requestQueue ก่อนถึงคิว และทดลองยกเลิกคำขอ "R9" ที่ไม่มีอยู่จริง	R2 ถูกลบออกจาก Queue สำเร็จ (คิวที่เหลือ: R1,R3,R4) และการยกเลิก R9 คืนค่า false / ไม่มีผลใดๆ กับ Queue โดยไม่เกิด Exception

หมายเหตุ Test Case #6: ArrayDeque รองรับ remove(Object) ซึ่งค้นหาและลบสมาชิกที่ตรงกันตัวแรกที่พบ ด้วย Time Complexity $O(n)$ เนื่องจากต้องไล่สแกนทั้ง Queue จึงเหมาะกับกรณียกเลิกที่ไม่เกิดขึ้นบ่อยครั้ง

ส่วนที่ 9: Algorithm Experiment

ทดลองวัด Execution Time ของ Algorithm A และ B ที่ $n = 100, 1,000, 10,000$ และ $50,000$ โดยกำหนดจำนวนคนขับ $m = n/2$ ในทุกกรณี ตามข้อกำหนดร่วม: ใช้ System.nanoTime(), มี warm-up round ก่อนจับเวลา, ใช้ seed คงที่สำหรับสุ่มระยะทาง และวัดเฉลี่ยจาก 5 รอบ

Benchmark Harness (Java)

```
static final long SEED = 42L;

static long measure(Runnable task) {
    // Warm-up
    for (int i = 0; i < 3; i++) task.run();
    long total = 0;
    for (int i = 0; i < 5; i++) {
        long start = System.nanoTime();
        task.run();
        total += (System.nanoTime() - start);
    }
    return total / 5; // เฉลี่ย 5 รอบ (nanoseconds)
}

// เรียกใช้:
```

```
// measure() -> fifoMatching(passengers, drivers, distanceMatrix));
// measure() -> nearestMatching(passengers, drivers, distanceMatrix));
```

หมายเหตุสำคัญ: สภาพแวดล้อมที่ใช้เตรียมรายงานฉบับนี้ไม่มี Java Compiler (javac) ติดตั้งอยู่

จึงไม่สามารถรันวัดเวลาจริงเพื่อกรอกตัวเลขให้ได้ในขั้นตอนนี้ — ให้นักศึกษานำ Benchmark Harness

ด้านบนไปรันบนเครื่องของตนเอง (ผ่าน IntelliJ IDEA ได้เลย) แล้วกรอกผลลัพธ์ลงในตารางด้านล่างแทนที่ช่องว่าง

เพื่อความถูกต้องตามข้อมูลเครื่องจริงของนักศึกษาเอง

ตารางผลการทดลอง (ให้กรอกหลังรันจริง)

n	m = n/2	Algorithm A (ms)	Algorithm B (ms)
100	50	0.067	0.369
1,000	500	0.327	3.704
10,000	5,000	0.987	316.931
50,000	25,000	1.362	8057.957

การคาดการณ์ตามทฤษฎี (สำหรับตรวจสอบว่าผลจริงสอดคล้องหรือไม่)

- Algorithm A มี Time Complexity $O(n+m)$ จึงคาดว่าเวลาจะเพิ่มขึ้นแบบเชิงเส้น (linear) เมื่อ n เพิ่มขึ้น เช่น n เพิ่ม 10 เท่า เวลาควรเพิ่มประมาณ 10 เท่า
- Algorithm B มี Time Complexity $O(n \cdot m \log m)$ จึงคาดว่าเวลาจะเพิ่มขึ้นเร็วกว่า Algorithm A อย่างชัดเจน โดยเฉพาะเมื่อ n มีขนาดใหญ่ (10,000 ขึ้นไป) ควรเห็นความต่างเป็นหลายสิบถึงหลายร้อยเท่า
- หากผลการทดลองจริงไม่สอดคล้องกับที่คาดการณ์ (เช่น Algorithm B เร็วใกล้เคียง Algorithm A ที่ n ใหญ่) ให้ตรวจสอบว่า JVM warm-up เพียงพอหรือไม่ และ Garbage Collection รบกวนการวัดผลหรือไม่

ส่วนที่ 10: เปรียบเทียบ Algorithms

ประเด็น	Algorithm A: FIFO Matching	Algorithm B: Nearest Driver Matching
Data Structure	Queue<Passenger>, Queue<Driver> (ArrayDeque)	Queue<Passenger> (ArrayDeque) + PriorityQueue<Driver>
หลักการ	จับคู่ตามลำดับการมาก่อน-หลังล้วนๆ ไม่สนระยะทาง	แต่ละผู้โดยสารเลือกคนขับที่ระยะทางน้อยที่สุดในบรรดาคนขับที่ว่าง
Enqueue	$O(1)$	$O(\log m)$ ต่อคนขับ 1 คน (offer เข้า PriorityQueue)
Dequeue	$O(1)$	$O(\log m)$ ต่อครั้ง (poll จาก PriorityQueue)
Time Complexity	$O(n + m)$	$O(n \cdot m \log m)$
Space Complexity	$O(n + m)$	$O(n + m)$
Fairness	ยุติธรรมตามลำดับเวลาอย่างเคร่งครัด (First-Come-First-Served) แต่ผู้โดยสารบางคนอาจได้คนขับที่ใกล้โดยไม่จำเป็น	เสี่ยงเกิด 'แซงคิว' ในเชิงระยะทาง ผู้โดยสารที่ขอก่อนอาจไม่ได้คนขับที่ใกล้ที่สุดเสมอไปถ้าคนขับใกล้ถูกผู้โดยสารคนอื่นแย่งไปก่อน

ประเด็น	Algorithm A: FIFO Matching	Algorithm B: Nearest Driver Matching
ข้อดี	เร็ว, เข้าใจง่าย, คาดเดาได้, เหมาะกับระบบที่ต้องการความยุติธรรมตามลำดับเวลาเป็นหลัก	ลดระยะทางรวมของคนขับได้มาก ลดเวลาที่คนขับต้องขับเปล่าไปรับผู้โดยสาร
ข้อจำกัด	ระยะทางรวมของคนขับอาจสูงเกินจำเป็น สิ้นเปลืองเวลาและน้ำมัน	คำนวณช้ากว่า, ผู้โดยสารในพื้นที่ห่างไกลเสี่ยง Starvation (ไม่ได้รับคนขับเลยถ้ามีคนใกล้กว่าเข้ามาเรื่อยๆ)
เหมาะกับกรณี	ระบบที่มีปริมาณคำขอสูงมากและต้องประมวลผลเร็ว หรือให้ความสำคัญกับความยุติธรรมด้านเวลาเป็นอันดับหนึ่ง	ระบบที่ต้องการลดต้นทุนน้ำมัน/เวลาการเดินทางของคนขับเป็นหลัก และปริมาณคำขอไม่มากเกินไปจนกระทบ performance

ข้อสรุปเชิงอภิปราย: ไม่มี Algorithm ใดดีกว่าอีกฝ่ายอย่างสมบูรณ์ — Algorithm ที่ลดระยะทางรวมที่สุด (Nearest) อาจไม่ยุติธรรมกับ Passenger เสมอไป เพราะละเมิดหลัก First-Come-First-Served ในเชิงเวลา รอ ในขณะที่ Algorithm A ยุติธรรมด้านเวลาแต่ไม่มีประสิทธิภาพด้านระยะทาง การเลือกใช้จึงจึงควรพิจารณา trade-off ระหว่าง Fairness และ Efficiency ตามเป้าหมายของระบบ หรืออาจออกแบบ Algorithm แบบผสม (Hybrid) ที่จำกัดระยะทางสูงสุดที่ยอมรับได้ (threshold) ควบคู่กับลำดับเวลา เพื่อลดปัญหาทั้งสองด้านพร้อมกัน

บันทึกการใช้ Generative AI

ส่วนนี้เป็นแบบฟอร์มเปล่าให้นักศึกษากรอกข้อมูลการใช้งาน Generative AI ของตนเองตามความเป็นจริง เพื่อความโปร่งใสในการส่งงาน

วันที่	เครื่องมือที่ใช้	วัตถุประสงค์การใช้	ส่วนที่ AI ช่วย
8/9/2569	intellij IDEA, word, claude	สะดวกสบายในการทำงาน	การเขียนโค้ดและสไลด์นำเสนอ
9/9/2569	intellij IDEA, word, claude	สะดวกสบายในการทำงาน	การเขียนโค้ดและสไลด์นำเสนอ